

# Informe Técnico

## Estructura de Proyecto y Data Binding en Ionic

Módulo 0488 · Desarrollo de Interfaces — RA1 (CE1.d) / RA2 (CE2.e)

**Alumno:** Héctor

**Fecha:** 21 de septiembre de 2026

**Entorno:** Ionic / Angular (proyecto "galeria")

### 1. Introducción y objetivo

El objetivo de esta actividad es comprender la arquitectura de dependencias de un proyecto Ionic/Angular, arrancar el entorno de desarrollo local y establecer una comunicación de datos unidireccional (Data Binding) entre el controlador (TypeScript) y la vista (HTML), utilizando recursos estáticos alojados localmente en el directorio de assets.

### 2. Paso 1 — Auditoría de entorno

Antes de comenzar el desarrollo es necesario comprobar que las herramientas del entorno están instaladas correctamente y conocer sus versiones, ya que la compatibilidad entre Node.js, Ionic CLI y Angular CLI condiciona el correcto funcionamiento del proceso de compilación. Desde la terminal integrada de VS Code se ejecutan los siguientes comandos:

```
node -v
ionic -v
ng version
```

 Captura 1

```
PS C:\Users\hecto\Documents\2DAM\DISEÑO DE INTERFACES\myApp> node -v
v24.19.0
PS C:\Users\hecto\Documents\2DAM\DISEÑO DE INTERFACES\myApp> ionic -v
7.2.1
PS C:\Users\hecto\Documents\2DAM\DISEÑO DE INTERFACES\myApp> ng version
```



```
Angular CLI      : 22.0.1
Angular          : 22.0.1
Node.js          : 24.19.0
Package Manager  : npm 11.17.0
Operating System : win32 x64
```

| Package                   | Installed Version | Requested Version |
|---------------------------|-------------------|-------------------|
| @angular/build            | 22.0.1            | 22.0.1            |
| @angular/cli              | 22.0.1            | 22.0.1            |
| @angular/common           | 22.0.1            | 22.0.1            |
| @angular/compiler         | 22.0.1            | 22.0.1            |
| @angular/compiler-cli     | 22.0.1            | 22.0.1            |
| @angular/core             | 22.0.1            | 22.0.1            |
| @angular/forms            | 22.0.1            | 22.0.1            |
| @angular/language-service | 22.0.1            | 22.0.1            |
| @angular/platform-browser | 22.0.1            | 22.0.1            |
| @angular/router           | 22.0.1            | 22.0.1            |
| rxjs                      | 7.8.2             | ~7.8.0            |
| typescript                | 6.0.3             | ^6.0.3            |

*Salida de terminal mostrando las versiones instaladas de Node.js, Ionic CLI y Angular CLI*

## Justificación técnica

Angular introduce cambios de API entre versiones mayores, y cada versión de Ionic CLI solo es compatible con rangos concretos de Angular y Node.js. Verificar las versiones en esta fase permite detectar posibles incompatibilidades antes de que provoquen errores de compilación más adelante.

**Versión de Node.js: 24.19.0**

**Versión de Ionic CLI: 7.2.1**

**Versión de Angular CLI: 22.0.1**

## 3. Paso 2 — Análisis de dependencias (package.json)

El archivo package.json es el manifiesto del proyecto: define metadatos, scripts de línea de comandos y, sobre todo, las dependencias necesarias para que la aplicación funcione.

Captura 2

```
"dependencies": {  
  "@angular/common": "22.0.1",  
  "@angular/compiler": "22.0.1",  
  "@angular/core": "22.0.1",  
  "@angular/forms": "22.0.1",  
  "@angular/platform-browser": "22.0.1",  
  "@angular/router": "22.0.1",  
  "@ionic/angular": "^9.0.0",  
  "ionicons": "^8.1.0",  
  "rxjs": "~7.8.0",  
  "tslib": "^2.3.0"  
},  
"devDependencies": {  
  "@angular/build": "22.0.1",  
  "@angular/cli": "22.0.1",  
  "@angular/compiler-cli": "22.0.1",  
  "@angular/language-service": "22.0.1",  
  "@ionic/angular-toolkit": "^13.0.0",  
  "ajv": "^8.20.0",  
  "angular-eslint": "22.0.0",  
  "eslint": "^9.16.0",  
  "jsdom": "^26.0.0",  
  "typescript": "^6.0.3",  
  "typescript-eslint": "^8.61.0",  
  "vitest": "^4.0.8"  
},  
"allowScripts": {}
```

Secciones "dependencies" y "devDependencies" del archivo package.json

## Diferencia entre dependencies y devDependencies

- dependencies: librerías necesarias para que la aplicación funcione en producción (por ejemplo, @angular/core, @ionic/angular, rxjs). Se incluyen en el bundle final que se despliega.
- devDependencies: herramientas usadas exclusivamente durante el desarrollo (compilador de TypeScript, linters, utilidades de testing). No se incluyen en el bundle final de producción, por lo que no aumentan el peso de la aplicación entregada.

**Versión exacta de Angular (@angular/core) usada en el proyecto:**

## 4. Paso 3 — Arranque del servidor de desarrollo

El comando `ionic serve` inicializa el servidor web de desarrollo local (por defecto en `http://localhost:8100`). Compila el código en memoria (Webpack o Vite, según la versión de Angular Builder) y abre un socket con el navegador:

ionic serve

 Captura 3

```
PS C:\Users\hecto\Documents\2DAM\DISEÑO DE INTERFACES> cd myApp
PS C:\Users\hecto\Documents\2DAM\DISEÑO DE INTERFACES\myApp> ionic serve
> ng.cmd run app:serve --host=localhost --port=8100
[ng] > Building...
[ng] One or more browsers which are configured in the project's Browserslist configuration fall outside Angular's browser support for this version.
[ng] Unsupported browsers:
[ng] chrome 110, chrome 109, chrome 108, chrome 107, edge 110, edge 109, edge 108, edge 107, firefox 111, firefox 110, firefox 109, firefox 108, firefox 107, firefox 106, ios_saf 16.3, ios_saf 16.2, ios_saf 16.1, safari 16.3, safari 16.2, safari 16.1
[ng] ✓ Building...
[ng] Initial chunk files | Names | Raw size
[ng] styles.css | styles | 57.70 kB |
[ng] main.js | main | 2.75 kB |
[ng] | Initial total | 60.45 kB
[ng] Lazy chunk files | Names | Raw size
[ng] home.page-OVHPQCHC.js | home-page | 4.42 kB |
[ng] Application bundle generation complete. [3.472 seconds] - 2026-09-21T11:10:59.639Z
[ng] NOTE: Raw file sizes do not reflect development server per-request transformations.
[ng] One or more browsers which are configured in the project's Browserslist configuration fall outside Angular's browser support for this version.
[ng] Unsupported browsers:
[ng] chrome 110, chrome 109, chrome 108, chrome 107, edge 110, edge 109, edge 108, edge 107, firefox 111, firefox 110, firefox 109, firefox 108, firefox 107, firefox 106, ios_saf 16.3, ios_saf 16.2, ios_saf 16.1, safari 16.3, safari 16.2, safari 16.1

[INFO] Development server running!

Local: http://localhost:8100

Use Ctrl+C to quit this process

[ng] → Local: http://localhost:8100/
[ng] → press h + enter to show help
[INFO] Browser window opened to http://localhost:8100!
```

*Salida de la terminal indicando el puerto de despliegue y el tiempo de compilación*

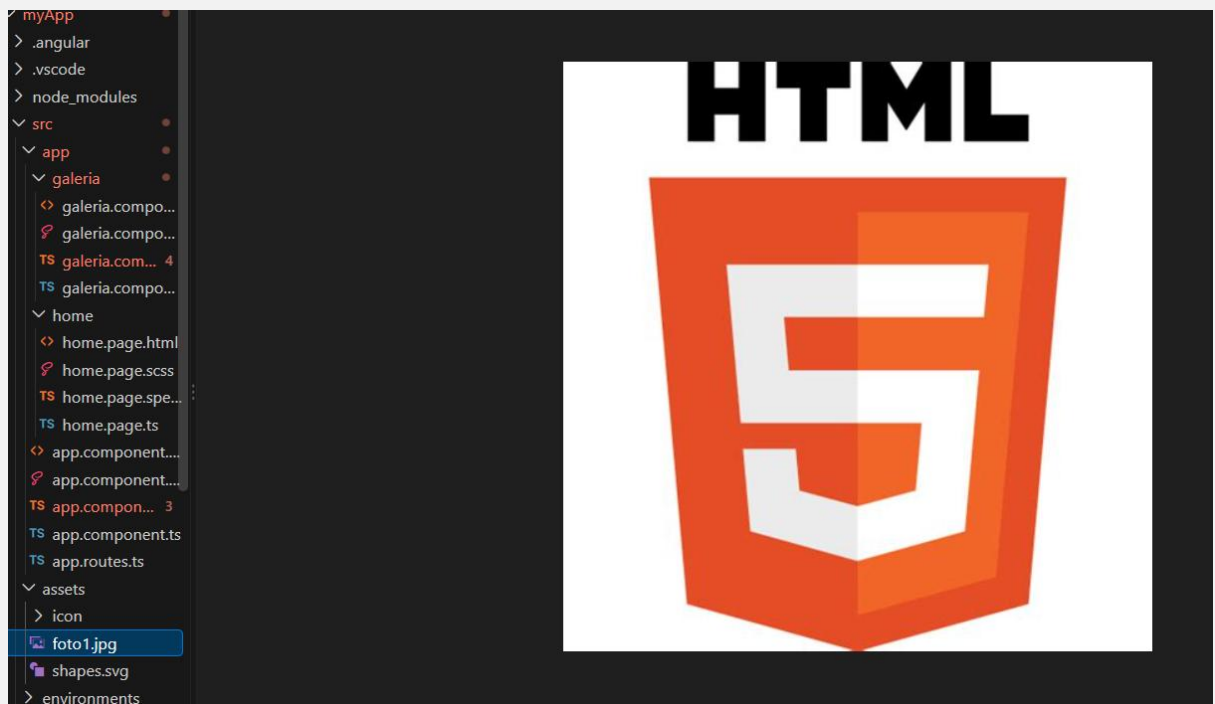
## Utilidad técnica del Live Reload

El Live Reload (Hot Module Replacement) inyecta automáticamente en el navegador los cambios guardados en el editor, sin recargar la página completa ni perder el estado actual de la aplicación. Esto acelera notablemente el ciclo de desarrollo, ya que evita tener que reconstruir y recargar manualmente la aplicación tras cada cambio.

## 5. Paso 4 — Gestión de assets

Las imágenes, fuentes y demás archivos estáticos no se importan como código: deben alojarse en el directorio `src/assets/`. Durante la compilación, el builder de Angular mapea ese directorio a la raíz del servidor, de modo que los recursos pueden referenciarse mediante rutas relativas directas (por ejemplo, `assets/foto1.jpg`).

 Captura 4



Árbol de directorios de VS Code mostrando la imagen añadida dentro de src/assets/

## 6. Paso 5 — Desarrollo del componente "galeria"

Partiendo del componente generado con ionic ng g c galeria, se declaran en el TypeScript las variables de estado y se consumen en la plantilla HTML mediante Data Binding.

### galeria.component.ts

Captura 5ª

```
myApp > src > app > galeria > TS galeria.component.ts > GaleriaComponent
1  import { Component, OnInit, CUSTOM_ELEMENTS_SCHEMA } from '@angular/core';
2
3  @Component({
4    selector: 'app-galeria',
5    templateUrl: './galeria.component.html',
6    styleUrls: ['./galeria.component.scss'],
7    standalone: true,
8    schemas: [CUSTOM_ELEMENTS_SCHEMA],
9  })
10 export class GaleriaComponent implements OnInit {
11   public tituloSeccion: string = 'Galería de Imágenes Dinámica';
12   public descripcion: string = 'Esta imagen se carga desde el directorio de assets mediante Property Binding.';
13   public rutaImagenLocal: string = 'assets/foto1.jpg';
14
15   constructor() {}
16
17   ngOnInit() {}
18 }
```

Código del archivo galeria.component.ts con las variables declaradas, tipadas e inicializadas

 **Captura 5b**

```
app > src > app > galeria > galeria.component.html > ...  
<ion-header>  
  <ion-toolbar>  
    <!-- Interpolación de la variable tituloSeccion -->  
    <ion-title>{{ tituloSeccion }}</ion-title>  
  </ion-toolbar>  
</ion-header>  
  
<ion-content class="ion-padding">  
  <!-- Interpolación para texto -->  
  <p>{{ descripcion }}</p>  
  
  <!-- Property Binding [src] apuntando a la variable rutaImagenLocal -->  
  <div class="image-container">  
    <img [src]="rutaImagenLocal" alt="Imagen cargada localmente" />  
  </div>  
</ion-content>
```

*Código del archivo galeria.component.html usando interpolación {{ }} y Property Binding [src]*

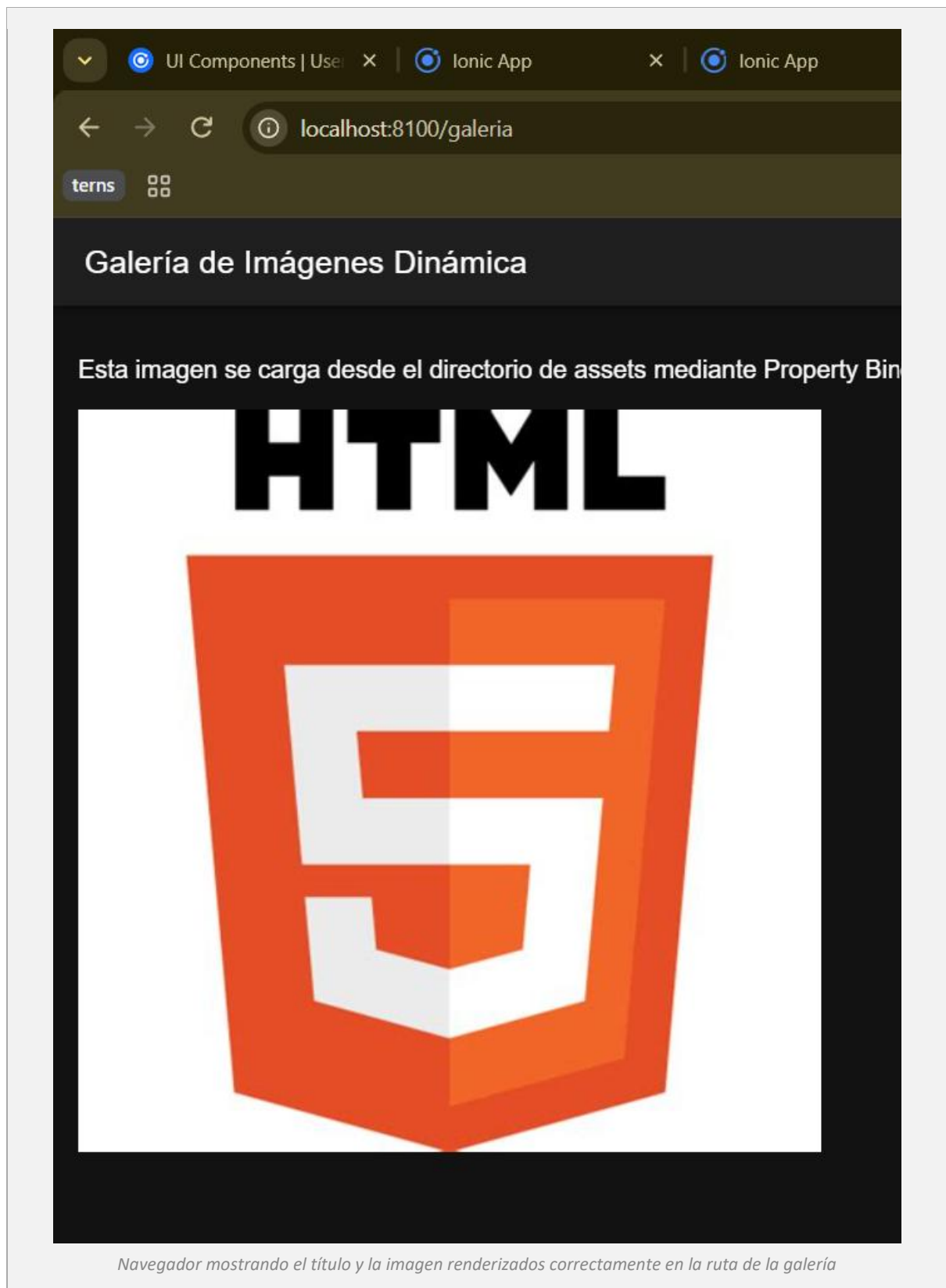
## Mecanismos de Data Binding utilizados

- Interpolación {{ }}: enlace unidireccional (one-way binding) que renderiza en el HTML el valor de una variable definida en el TypeScript.
- Property Binding [propiedad]="variable": asigna dinámicamente el valor de una variable a un atributo HTML estándar, como el src de una imagen.

## 7. Paso 6 — Validación final

Por último, se accede desde el navegador a la ruta correspondiente del componente galeria para comprobar que el título y la imagen se renderizan correctamente a partir de las variables definidas.

 **Captura 6**



## 8. Conclusión

Esta actividad ha permitido comprobar cómo se organiza un proyecto Ionic/Angular a nivel de dependencias, cómo se pone en marcha el entorno de desarrollo aprovechando el Live Reload para

iterar rápidamente, y cómo se establece un flujo de datos unidireccional entre el controlador y la vista mediante interpolación y Property Binding, usando además recursos estáticos alojados en el directorio de assets.